Abgabe 5: Projekt 2 (Wetterstation)

Abzugeben sind bis Sonntag nach der mündlichen Abnahme, um 23:59 Uhr:

- eine in Ihrme Git Repository hinterlegte PDF-Datei mit dem Namen `abgabe5.pdf`, in der die unten *kursiv* angegebenen Fragen beantwortet werden. Sie haben hier die Freiheit, ein beliebiges Tool zur Erstellung der PDF zu benutzen, wobei wir Ihnen natürlich LaTeX empfehlen würden.
- der in Ihrem Git Repository mit dem Tag `abgabe_5` eingecheckte Code in dem CubeIDE-Projekt **P2_Weatherstation** zu den praktischen Aufgaben (nicht kursiv)

I^2C-Protokoll



Benutzen Sie für die Beantwortung der folgenden Fragen zusätzlich noch das in den Ressourcen zur Verfügung gestellte Dokument **Understanding_the_I2C_bus.pdf**

1. *Wann kann ein Datentransfer auf dem Bus initiiert werden? Welchen Zustand müssen die beiden Leitungen dafür haben?*
2. *Was stellt eine **Repeated Start Condition** dar und wofür ist sie nützlich?*
3. *Wann generiert ein Empfänger ein **Not Acknowledge (NACK)** auf der Leitung?*
4. *Wie läuft der Schreibprozess von Master zu Slave ab? Welche beiden Bytes müssen geschickt werden, bevor die eigentlichen Datenbytes folgen?*
5. *Worin unterscheidet sich ein Leseprozess von dem soeben beschriebenen Schreibprozess?*

BME280-Umgebungssensor

1. *Bevor Sie sich um die Implementierung kümmern, muss der Sensor korrekt an ihr Board angeschlossen werden. Benutzen Sie das Datenblatt des BME280 [1], um folgende Fragen zu beantworten:*
   - *Mit welcher Spannung kann der Sensor betrieben werden? Wo müssen Sie also die V_{CC}-Leitung an Ihrem Board anschließen?*
   - *Sie sollen die I2C1-Peripherie Ihres Mikrocontrollers verwenden. Wo schließen Sie also die Leitungen SDA und SCL an?*
   - *Welche Bedeutung haben die beiden restlichen Leitungen CS und ADDR (im Datenblatt auch CSB und SDO genannt) bei Benutzung des I^2C-Interfaces? Wo schließen Sie diese an?*
2. *Welche Funktionen bietet Ihnen die HAL an, um im Polling-Mode über I^2C zu schreiben und zu lesen? Worin unterscheiden sich die beiden Arten im Hinblick auf die von Ihnen oben in den Aufgaben 4 und 5 beschriebenen Schreib- und Leseprozesse des I^2C-Protokolls?*
3. *Wie initialisieren Sie mithilfe der Bosch-Bibliothek den Sensor? Wie teilen Sie der Bibliothek mit, wo und wie der Sensor ausgelesen bzw. zu ihm Daten gesendet werden können?*



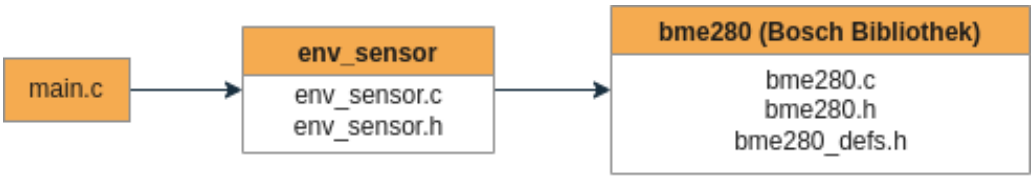
Bei der Initialisierung des Sensors durch die Bibliothek werden schon Schreib-

bzw. Leseoperationen auf dem Sensor ausgeführt. Somit muss die benötigte I^2C- und GPIO-Peripherie des Mikrocontrollers schon vorher konfiguriert und initialisiert werden!



Das Feld `delay_us` der Initialisierungs-Struktur wird in der README der Bibliothek fälschlicherweise als `delay_ms` angegeben. Es wird eine Delayfunktion erwartet, die eine Verzögerungszeit in **Mikrosekunden** entgegennimmt.

4. *In welchen Einheiten liefert die Bosch-Bibliothek die unterschiedlichen Sensordaten?*
5. Entwickeln Sie ein Modul (z.B. *env_sensor*), welches für die main als weitere Abstraktion zu der von Bosch zur Verfügung gestellten Bibliothek fungiert (siehe Abbildung). **Orientieren** Sie sich bei dem konkreten Auslesen der drei Werte (Temperatur, Luftdruck und -feuchtigkeit) an dem Beispiel "Stream sensor data in forced mode" in der README der Bibliothek.



6. Testen Sie Ihr Modul, indem Sie die Sensordaten auf dem LCD ausgeben.

CAN-Protokoll

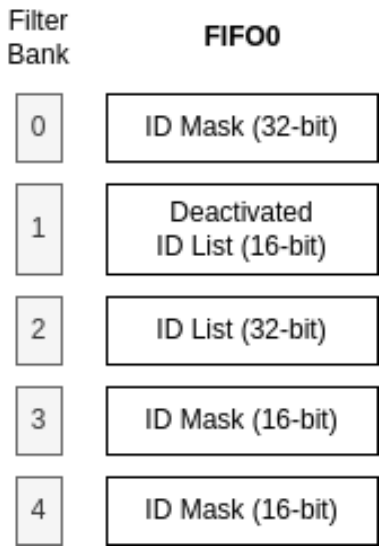
1. *Welche Arten von Kommunikations-Frames gibt es beim CAN? Wofür werden diese verwendet?*
2. *Wie ist ein Daten-Frame aufgebaut?*
3. *Um Kollisionen (gleichzeitiger Buszugriff) aufzulösen, verwendet CAN ein CSMA/CR-Verfahren. Welches Bit stellt hierbei das dominante und welches das rezessive dar? Welche Auswirkung hat dies auf die Priorisierung mittels des Identifiers einer Nachricht?*
4. *Was stellt das Bit Stuffing beim CAN dar?*

CAN-Interface des Cortex-M4

1. *Welche drei Testmodi werden für das CAN-Interface angeboten? Was sind ihre Unterschiede?*
2. *Eine Filterbank wird im Mask Mode und mit einer 16-Bit-Skalierung konfiguriert. Register **CAN_FiR1** hat den Wert `0xFE00AA00` und **CAN_FiR2** den Wert `0xFE00D600`.*
   a. *Betrachten Sie das Mapping in Abbildung 342 des Reference Manuals. Welche Bits einer eingehenden Nachricht würden mit den definierten Masken geprüft werden?*
   b. *Würden Nachrichten mit folgenden Standard-Identifiern diese Filterbank passieren und wenn ja, welcher der beiden eingestellten Filter der Filterbank wäre dafür verantwortlich?*
      i. `0b11000110101`
      ii. `0b10101011001`
      iii. `0b11010111111`
   c. *Geben Sie für diese Konfiguration das entsprechend gefüllte `CAN_FilterTypeDef` an. Nehmen Sie*

sich für das korrekte Befüllen der Mask- und Identifier-Felder die Implementierung der Funktion `HAL_CAN_ConfigFilter()` *in* ***stm32f4xx_hal_can.c*** *zur Hilfe. Dort finden Sie nämlich die Zuordnung dieser Struct-Felder auf die Register* ***CAN_FiR1*** *und* ***CAN_FiR2****.*

3. *Betrachten Sie folgende Konfiguration der Filterbänke:*



   Wie viele unterschiedliche Filter bildet diese Konfiguration ab? Ergänzen Sie die Ansicht um die Filter-Nummern (wie in Abbildung 343 des Reference Manuals).

4. *Welche Felder bietet der Struct-Typ* `CAN_TxHeaderTypeDef` *an? Was legen sie fest? Ignorieren Sie das Feld* `.TransmitGlobalTime` *, es wird nur im "Time triggered communication mode" benötigt.*
5. *Welche beiden Funktionen (ISR und Callback) muss man implementieren, wenn man auf Interrupts bezüglich des Eintreffens einer Nachricht in FIFO 0 reagieren möchte?*

Wetterstation

In diesem Projekt soll ein Netzwerk aus Wetterknoten aufgebaut werden. Jeder Wetterknoten misst die lokalen Wetterdaten Temperatur, Luftdruck und Luftfeuchtigkeit. Jede Praktikumsgruppe entwickelt ihren eigenen Wetterknoten und klinkt sich damit in das CAN-Netz ein, um dort die gewonnenen Wetterdaten zu publizieren.

Anforderungen:

- Der Standard-Identifier der CAN-Nachrichten **muss** folgenden Aufbau haben. Hält sich ein Knoten nicht daran, so kann er weder Nachrichten anderer Knoten interpretieren, noch können andere Knoten seine Nachrichten verstehen:



  Die Knoten-ID ist die Nummer Ihrer Gruppe und identifiziert Ihr Entwicklungsboard im Netz, die Sensor-ID können Sie aus folgender Tabelle erkennen:

Sensor-ID	Sensor-Datenformat
0	Temperatur in 0.01°C, `int16_t`, aufgeteilt auf zwei Bytes
1	Luftdruck in Pa, `uint32_t` aufgeteilt auf vier Bytes
2	Luftfeuchtigkeit in 0.01%, `uint16_t` aufgeteilt auf zwei Bytes

Für die Byte-Reihenfolge beim Versenden gilt **Big-endian**.

- Die Sensordaten sollen im Sekundentakt über den CAN-Bus veröffentlicht werden.
- Die gesamte Kommunikation über das CAN soll mit einer Baudrate von 125 kBit/s ablaufen.
- Richten Sie pro Sensordaten-Art eine Filterbank ein, welche eingehende Nachrichten eines beliebigen Knoten mit dieser Sensor-ID durchlässt.
- Auf dem LCD sollen neben der Knoten-ID die zu diesem Knoten gehörenden aktuellen Sensordaten dargestellt werden. Zu Beginn der Anwendung soll dies der eigene Knoten sein.
- Mithilfe des schon in Abgabe 1 verwendeten Joysticks soll zwischen der Ansicht der einzelnen Knoten gewechselt werden. Gehen Sie bei der Knotenanzahl von der Anzahl der Gruppen in diesem Semester aus.
- Setzen Sie bei der LCD-Anzeige Ihre Kreativität ein. Möglich wären beispielsweise Bargraph-Anzeigen für die einzelnen Sensorwerte, Scroll-Balken, etc.



Wir legen Ihnen sehr ans Herz, die zur Verfügung stehenden Testmodi des CAN-Interfaces zu verwenden, bevor Sie sich am echten CAN-Bus einklinken. Bei Verwendung des **Loopback-Modus** empfiehlt sich ebenfalls, den Logic-Analyzer zu verwenden, um am CANTX-Pin die herausgehenden Nachrichten zu validieren. Benutzen Sie hier den von der Logic-Analyzer-Software zur Verfügung gestellten CAN-Decoder.

[1] **BME280_datenblatt.pdf** im Ressourcen-Ordner